

Introduction to Intelligent Agents

1 What is AI?

The primary goal of (weak) artificial intelligence is to *build* intelligent entities. A related (but not a necessary) goal is to *understand* intelligent entities, and perhaps even to understand and engineer human intelligence (strong AI).

But what are *intelligent* entities? Russell and Norvig put forth four possible definitions, the second and third of which might be categorized as strong AI, and the first and fourth of which are more along the lines of weak AI:

1. systems that act like humans
2. systems that think like humans
3. systems that think rationally
4. systems that act rationally

	Human	Rational
Think	Thinking humanly: GPS	Thinking rationally: Logic
Act	Acting humanly: Turing Test	Acting rationally: Modern AI

Table 1: Four approaches to agent design. Following Russell and Norvig, 2020.

1.1 Acting Like Humans

In 1950, Turing proposed an operational definition of intelligence, later named the Turing Test [13]. Generalizing Turing, an entity could be deemed intelligent if it demonstrates an ability to achieve human-level performance in all cognitive tasks. Turing's test, specifically, was an imitation game, where the entity was required to fool an interrogator in conversation. Turing predicted that by the year 2000, computers would pass his test. More specifically, he predicted that the average interrogator would not be able to distinguish a computer from a human more than 70 per cent of the time, after a five minute conversation.

ELIZA, an early AI system developed by Weizenbaum [15], fails the Turing Test.

```
PERSON: My wife says I'm often depressed.
ELIZA: I'm sorry to hear you are depressed.
PERSON: The truth is I am quite unhappy.
ELIZA: Do you think coming here will help you not to be unhappy?
PERSON: Well, perhaps by talking to you I will find out why I hate my brother.
ELIZA: Tell me more about your family.
```

Like the word `brother` in this example, a sentence such as `I admire Mother Teresa` triggers the response `Tell me more about your family`.

In a 2025 study, Jones and Bergen played the imitation game with GPT-4.5 and human subjects [5]. GPT-4.5 fooled the interrogators about 73% of the time—they said GPT-4.5 was the human, not the human itself!—while ELIZA, which served as a baseline, fooled them less than a quarter of the time. The former percentage is significantly above chance, while the latter is significantly below.

Remark Weizenbaum showcased ELIZA as a Rogerian (i.e., client-centered, non-directive) therapist, but was then dismayed when users trusted ELIZA like they would a real therapist. People were surprisingly willing to attribute understanding even to this simple pattern-matching machine. This example resembles how AI adoption is progressing today. Here is a particularly pernicious example: a user describes an uncertain situation to an AI; the AI generates a plausible explanation; the user finds the explanation compelling, and provides the AI with more supporting evidence; the AI incorporates this additional evidence into the story; the user becomes more confident. This behavior is pervasive today because today’s AI are often sycophantic: they flatter their users rather than critique them, and can easily lead them down the garden path. Weizenbaum identified this potential risk early on, and argued that not matter how skilled computers become, choices requiring human judgment should not be delegated to them [16].

1.2 Thinking Like Humans

In 1959, Newell, Shaw, and Simon designed the General Problem Solver (GPS) [8], which they built to simulate human problem-solving strategies across a variety of domains. The name GPS derived from the program’s architecture, which distinguished between general knowledge about reasoning and specific domain knowledge. GPS used means-ends analysis in its search for solutions, computing the difference between the goal and the current state, and then attempting to minimize that difference. By comparing GPS traces with those of human subjects, Newell and Simon [9] concluded that the behavior of GPS resembled that of humans, broadly speaking, although both did things the other did not. GPS was one of the earliest programs used to test a theory of cognition, which today characterizes the field of cognitive science.

1.3 Thinking Rationally

The *Laws of Thought* approach to AI relies on patterns for argument structure rooted in Aristotle’s syllogisms (e.g., All men are mortal; Socrates is a man; therefore, Socrates is mortal). A formal logic is a way of capturing such reasoning patterns that depend on form alone, so that they can be applied regardless of their subject matter. The hope, then, is that if an agent’s basic knowledge can be expressed in a formal logic, everything else it needs to know can be derived mechanically—by the turn of a crank. (A brief history of attempts to place all of mathematics on such a footing, and why they failed, appears in Appendix A.)

Logic has been central to AI since its earliest days. In 1956, Newell, Shaw, and Simon’s Logic Theorist proved 38 of the first 52 theorems in Chapter 2 of Whitehead and Russell’s *Principia Mathematica* [9]. In 1958, McCarthy envisioned a system he called Advice Taker, in which commonsense knowledge would be represented as logical statements and an agent endowed with such a knowledge base would deduce the implications of its knowledge to figure out what to do [6]. In 1965, Robinson defined the resolution rule of logic, a *single* rule of logical inference that mimics proof by contradiction—the negation of a claim is added to a knowledge base, and then the rule is applied repeatedly, until a contradiction is found, at which point the claim is proven true—thereby greatly simplifying the search for proofs, which heretofore had required a search over multiple inference rules [11]. The Prolog—PROgramming in LOGic—programming language was developed shortly thereafter, the key idea being to write programs as logical statements and to execute them via inference, i.e., proof search via resolution [1]. Logic remains in widespread use today: e.g., SAT solvers, to verify hardware; STRIPS-style planners for robot task planning (e.g., first walk to the table, then

pick up the cup, etc.); and proof assistants, such as Lean, paired with large language models (LLMs), to assist mathematicians in proving theorems, and to compete successfully in International Math Olympiad.

Despite its successes, the logical approach faces two main obstacles. First and foremost, classical logic is monotonic: new premises can never invalidate an established conclusion. Suppose, for example, that an agent’s knowledge base contains the sentences “Birds fly” and “Tweety is a bird.” The logical conclusion is that “Tweety flies.” If, however, Tweety is a penguin, then Tweety doesn’t fly. This logical contradiction in a knowledge base entails everything, i.e., the knowledge base explodes! Moreover, to try to encode the penguin exception and all others in an agent’s knowledge base is futile (penguins don’t fly; birds with injured wings don’t fly; birds in cages don’t fly; etc.), and even if all exceptions could be encoded, it suddenly becomes impossible to prove that the cardinal perched high on the tree branch in my backyard flies, because we first have to establish that it isn’t a penguin, and then that it doesn’t have a broken wing, and then that it isn’t in a cage, and so on—information that may not be available. The alternative is to assume by default that none of the exceptions hold, and then, when we discover that Tweety is a penguin, to retract the earlier conclusion that Tweety flies in favor of its negation. This is the agenda of non-monotonic logic.

The second difficulty with the logical approach is that much of our so-called knowledge is in fact uncertain: e.g., a cough is probably a cold, but it could be tuberculosis. Modern AI models reasoning under uncertainty by using probability to represent degrees of belief. If instead of ascribing 100% to the sentence “Tweety flies,” we leave some room for error, we can later adjust that probability upon observing additional evidence. In short, logic remains a key piece of the AI puzzle, but it is not the whole story.

1.4 Acting Rationally

Modern AI can be characterized as the engineering of **rational agents**. An **agent** is an entity that (i) perceives, (ii) reasons, and (iii) acts. In computational terms, that which is perceived is an *input*; to reason is to *compute* or to *learn*; and to act is to *output* the result of computation.

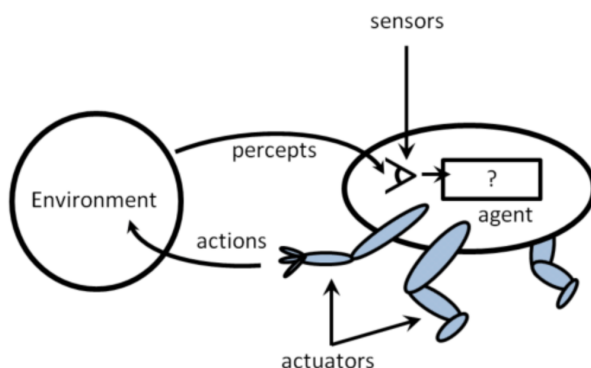


Figure 1: Intelligent Agents = Perception + Reason + Actuation. [Russell & Norvig, 2020]

Examples of agents include human agents, robotic agents, and software agents. (See Table 2.)

A typical agent maintains a **world model**, which is its representation of its knowledge and beliefs about the world, and updates its world model in light of new information.

Agents are endowed with **objectives** by their designers. A **rational** agent is one that acts to maximize those objectives, given its current world model. Most people are in fact **boundedly rational**: they are “satisficers, not optimizers, meaning they settle for decisions that are good enough [12];

Agent	Sensors	Actuators
Human	Senses	Arms, Legs, etc.
Robotic	Cameras	Motors, Wheels, etc.
Software	Bit Strings	Bit Strings

Table 2: Examples of Agents.

Agents [18] are often distinguished from typical computational processes by their **autonomy**—they operate without direct human intervention. In addition, agents are both **reactive**—they perceive their environments, and attempt to respond in a timely manner to changing conditions—and **proactive**—their behavior is goal-directed, rather than simply response-driven. Finally, agents in a multiagent system also need **social** skills.

Autonomous agents may be **rule-based**, **goal-based**, or **utility-based**. Rule-based agents operate according to hard-coded sets of rules, like ELIZA. A goal-based agent acts so as to achieve its goals, by planning a path from its current state to a goal state, like GPS or theorem provers. Utility-based agents distinguish between goals, based on utilities that are associated with goal states.

Learning is yet another aspect of agent design. A **learning** agent’s performance improves with experience [7].

2 Shakey the Robot

Shakey the robot, built by SRI (Stanford Research Institute), was the first general-purpose mobile robot. It was equipped with what has evolved to be typical components of an autonomous agent:

- Sensors (e.g., a camera) to gather data about its environment (e.g., objects in the room, walls, etc.)
- Communication channels (e.g., a wireless radio antenna) to communicate with a mainframe computer, because its onboard computer was insufficient for the automated reasoning it employed.
- Automated reasoning capabilities: it used STRIPS (Stanford Research Institute Problem Solver) [3], an early planner that employed a resolution theorem prover and means-ends analysis *à la* GPS to break down its high-level goals into a plan that comprised smaller subgoals.
- Actuators: Shakey could pan its camera around the room to collect data about its environment, moved itself by rolling around on wheels, and moved objects with a push bar.

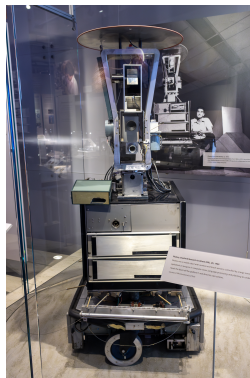


Figure 2: Shakey the Robot. Image source: Wikipedia.

3 Agent Environments

Agent environments may be characterized at least partially as follows:

- Deterministic vs. Nondeterministic: is the next state predictable (e.g., chess), or is there uncertainty about state transitions (e.g., backgammon)?
- Discrete vs. Continuous: can the environment be described in discrete terms (e.g., chess), or is the environment continuous (e.g., driving)?
- Static vs. Dynamic: is the environment static (e.g., chess), or can it change while the agent is reasoning about its plan of action (e.g., driving)?
- Sequential vs. One-shot: does the agent need to reason about the future impact of its immediate actions (e.g., chess), or can it treat each action independently (e.g., *Rochambeau*)?
- Fully observable vs. Partially observable: do agents observe the full state of the world, as in chess, or only partial information about the state, as in poker?
- Single agent vs. Multiagent: can we assume the agent is alone in its environment, or must it explicitly reason about the actions of other agents (e.g., chess, backgammon, *Rochambeau*, driving)?

4 Subfields of AI

The subfields of artificial intelligence can be classified in terms of their role in either perception, reasoning, or actuation.

- Perception
 - computer vision
 - speech recognition
 - natural language processing
- Reasoning (i.e., problem solving): mapping from percepts to actions
 - planning
 - automated reasoning
 - knowledge representation and belief updating
 - search and optimization
 - decision/game theory
 - machine learning
- Actuation
 - robotics
 - software agents

Some (by now) historic¹ examples of AI systems, besides our friend Shakey, include the following, described in terms of their percepts, reasoning mechanisms, and actuators.

- Xavier, the mail delivery robot, developed at CMU
 - Perception: vision, sonar, web interface
 - Reasoning: A* search, Bayes classification, hidden Markov models
 - Actuation: wheeled robotic actuation
- Pathfinder, the medical diagnosis system, developed by Heckerman during his Ph.D. at Stanford
 - Perception: input symptoms and test results
 - Reasoning: Bayesian networks, Monte-Carlo simulations
 - Actuation: output diagnoses and further test suggestions
- TDGammon, a world-class backgammon player, built by Gerry Tesauro of IBM Research
 - Perception: keyboard input
 - Reasoning: reinforcement learning, neural networks
 - Actuation: graphical output shows dice and movement of pieces
- ALVINN, the automated driver, developed by Pomerleau at CMU
 - Perception: video camera
 - Reasoning: neural networks and hand-engineered solutions
 - Actuation: land vehicle controller to turn the steering wheel
- PROVERB, a world class crossword puzzle solver, developed by Michael Littman and his students at Duke University²
 - Perception: grid, clues, background databases
 - Reasoning: belief net inference and “turbo decoding”
 - Actuation: filling in the grid

5 Other Definitions of AI

Rich defined AI as “the study of how to make computers do things at which, at the moment, people are better.” [10] Related, Tesler called intelligence “whatever machines haven’t done yet.” Others went on to define the “AI effect” based on this observation: before computers can perform a task, it is suspected that it requires “real” intelligence, but once computers succeed, the goalposts move—that activity is no longer believed to require real intelligence.

AI is the design of flexible programs that respond productively in situations that were not specifically anticipated by the designer [2].

AI is the construction of computations that perceive, reason, and act effectively in uncertain environments. In this definition, the psychological aspects of AI are perception, reason, and action, and the “construction of computations” encompasses the computer science aspect of AI [17].

¹These notes were written in the early 2000s.

²Michael Littman is now the (first) AI Provost here at Brown University (and perhaps also the first in the world).

6 What if we succeed?

Here's what Woody Allen has to say: "My father lost his job because his plant bought a machine that is capable of doing everything my father could do . . . it wasn't so bad, until my mother went out and bought one as well."

A Logicism and Formalism

Formal logic was developed by Boole, Peano, Frege, and others in the mid-1800s, and then, near the turn of the century, the **logicism** school was advanced by Frege, Russell, and Whitehead and others. Logicism held that mathematics could be reduced to formal logic, so that all theorems could be deduced from axioms (i.e., self-evident truths, such as a proposition cannot be both true and false). Frege's system rested on such an axiom, namely every concept defines a collection. For example, the concept "red" defines the collection of all red things. In 1902, Russell asked Frege to consider the property "is *not* a member of itself," i.e., the set $R = \{x \mid x \notin x\}$. The difficulty with this property is, if $R \in R$, then R fails its defining property, i.e., " $R \in R$ iff $R \notin R$ ". In other words, Frege's foundation supports an inherent contradiction. Russell's paradox, as it came to be called, dealt logicism a severe blow.

Hilbert, a preeminent German mathematician, who is perhaps best known for the ten problems he posed in Paris in 1900, was a proponent of a school of thought known as **formalism**. Like logicism, which aimed to show that mathematics is logic in disguise and thus sound, Hilbert's program also aimed to put mathematics on solid footing by showing how all mathematical truths could be derived from a formal system of symbols, axioms, and mechanistic rules. The conjecture, in other words, was that a simple formal system of mathematics is complete (every truth is provable) and consistent (no contradictions are implied). Gödel's two incompleteness theorems shattered all hopes of the formalist school succeeding. His first theorem debunked completeness: any consistent system built on basic axioms and capable of basic (e.g., Peano) arithmetic necessarily contains true statements that the system itself cannot prove. His second pertained to consistency: no sufficiently strong and consistent mathematical system (e.g., Peano arithmetic) can prove its own consistency using only its own rules.

Related, in 1928, Hilbert and Ackermann posed the *Entscheidungsproblem*, which asks whether there exists a mechanical procedure to decide whether a statement of first-order logic is provable [4]. In 1936, Turing solved this problem by way of the **halting problem**,³ as did Church, via another means.⁴ The halting problem asks whether a computer program, which takes as input a computer program and an input to that program, can decide whether the input program will halt on its input or run forever [14]. To analyze the halting problem, Turing invented a formal model of computation, which today is known as the **Turing machine**. He then proved the halting problem is **undecidable**, which means that there is not and can never be (assuming his model of computation) a computer program that takes as input the question and then outputs the correct answer to the question on all inputs. In other words, it is not engineering limitations that render an undecidable problem unsolvable; rather, the bottleneck is logical. Turing then applied his understanding of the halting problem to show that likewise, the *Entscheidungsproblem* is undecidable, thereby dealing the final blow to Hilbert's formalist agenda.

³The modern name of this problem, namely "the halting problem", has been credited to the logician Martin Davis.

⁴Whereas Turing invented Turing machines to solve the *Entscheidungsproblem*, Church proceeded via the λ -calculus, an alternative model of computation developed with his student, Kleene. The λ -calculus forms the basis of functional programming. The Church-Turing thesis states that the λ -calculus and Turing machines both represent every "effectively calculable" function that humans could, with infinite resources, carry out using paper, pencil, and mechanical rules, without any creative intuition.

References

- [1] Alain Colmerauer, Henry Kanoui, Philippe Roussel, and Robert Pasero. Un système de communication homme-machine en français. Rapport de fin de contrat, Groupe de recherche en Intelligence Artificielle, Faculté des Sciences de Luminy, Université d'Aix-Marseille II, Marseille, France, 1973.
- [2] Thomas Dean, James Allen, and Yannis Aloimonos. *Artificial Intelligence: Theory and Practice*. Benjamin/Cummings, Redwood City, CA, 1995.
- [3] Richard E. Fikes and Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.
- [4] David Hilbert and Wilhelm Ackermann. *Grundzüge der theoretischen Logik*, volume 27 of *Grundlehren der mathematischen Wissenschaften in Einzeldarstellungen*. J. Springer, Berlin, 1928.
- [5] C. R. Jones and B. K. Bergen. Large language models pass a standard three-party Turing test. *Proceedings of the National Academy of Sciences*, 123(21):e2524472123, 2026.
- [6] John McCarthy. Programs with common sense. In *Mechanisation of Thought Processes: Proceedings of the Symposium of the National Physical Laboratory*, volume 1, pages 75–91, London, 1959. Her Majesty's Stationery Office.
- [7] Tom M. Mitchell. *Machine Learning*. McGraw-Hill Series in Computer Science. McGraw-Hill, New York, NY, 1997.
- [8] Allen Newell, J. C. Shaw, and Herbert A. Simon. Report on a general problem-solving program. In *Proceedings of the International Conference on Information Processing*, pages 256–264, Paris, 1959. UNESCO. Proceedings published 1960.
- [9] Allen Newell and Herbert A. Simon. The logic theory machine—a complex information processing system. *IRE Transactions on Information Theory*, 2(3):61–79, September 1956.
- [10] Elaine Rich. *Artificial Intelligence*. McGraw-Hill, New York, 1983.
- [11] J. Alan Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, January 1965.
- [12] Herbert A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118, 1955.
- [13] Alan M. Turing. Computing machinery and intelligence. *Mind*, 59(236):433–460, 1950.
- [14] Alan Mathison Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1937.
- [15] Joseph Weizenbaum. ELIZA—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1):36–45, jan 1966.
- [16] Joseph Weizenbaum. *Computer Power and Human Reason: From Judgment to Calculation*. W. H. Freeman and Company, San Francisco, 1976.
- [17] Patrick Henry Winston. *Artificial Intelligence*. Addison-Wesley, Reading, MA, 3rd edition, 1992.
- [18] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.